Lab 04 — D3.js: Data Join, Transitions, Enter & Exit

AI-Era Practical Laboratory | Data Science and Data Visualization

International University

Work mode: Groups of 2 | **In-class time:** 150 minutes (includes 20-minute introduction) | **Homework:** optional extension, up to 3 hours/group | **Total:** 100 points
Main topic: *D3 v7 data joins, transitions, keyed identity, chart updates and verification*

1. Learning objectives

After this lab, you should be able to:

- **LO1 — Reason about data joins:** distinguish *enter / update / exit* when the dataset grows, shrinks or is reordered, and explain why stable keys matter.
- **LO2 — Build a dynamic visualization:** create a horizontal bar chart of Vietnamese provincial GRDP indicators, with axes, data labels and working **Add**, **Remove** and **Sort** controls.
- **LO3 — Apply purposeful animation:** animate bar widths and positions, update axes and labels, and use transition lifecycle events (`start`, end) for visual feedback.
- **LO4 — Verify and debug:** use actual counts, identities, boundaries and timing to detect wrong data joins, misleading axes and animation failures.
- **LO5 — Adapt and own:** implement a small new requirement and explain or modify your pair's code during the Group Check.

Main bottleneck: Can you keep a chart truthful and its objects *identifiable* when data and user-selected ordering change? Animation alone is not proof of correctness.

2. Learning workflow and time

Stage	Minutes	AI policy	Deliverable
Instructor introduction	20	Listen / discuss	Predictions from pre-lab
THINK	**20**	**AI OFF**	Think sheet and update design
BUILD	**55**	**AI ON**	Working D3 chart
VERIFY	**20**	**AI ON**	Evidence from 5 tests + defect fix
ADAPT	**15**	**AI ON**	Undo-last-removal feature
GROUP CHECK	**15**	Instructor-defined	Pair explains + small live change
Buffer	5	—	Cleanup/submission
Total	**150**		

The pre-lab (15–20 minutes) should be completed **before class**. It is not part of the 150 minutes.

Pair roles: One **Driver** operates the keyboard; one **Navigator/Reviewer** checks decisions and proposes tests. Switch roles at least once. Both students must be able to explain the final implementation.

AI rule: THINK -> AI -> VERIFY. Do not use AI during THINK. AI may help with D3 syntax, debugging, review or alternative implementations during BUILD/VERIFY/ADAPT, but you must inspect and test its output. Record 2–3 meaningful AI interactions (at least one should be verification/review).

3. Scenario — Changing provinces, changing chart

A data team needs a dashboard showing the **GRDP indicator** of provinces in Viet Nam. Users initially see the **first 20 valid provincial records in source-file order**. They can add the next record, remove the most recently included record, or sort the **presentation order** without losing track of which province is which.

Dataset: https://tungth.github.io/data/vn-provinces-data.csv. The chart continues the provincial-data context of Lab 03. In the original handout, the heading incorrectly used **GRPD**; use **GRDP**, the Gross Regional Domestic Product indicator. **Inspect the CSV and confirm the exact column, reported units, and reference year before labeling the axis.** Do not present the source as current administrative boundaries or current-year statistics without checking its provenance.

Requirements preserved from the original Lab 04

1. A **horizontal bar chart** of 20 initial provinces with numeric labels at each bar's right side.
2. Axis with title, readable ticks and appropriate quantitative scale.
3. **Add province:** append the next unused valid province from the original data source.
4. **Remove province:** remove the last included province, not necessarily the visually bottom bar after sorting.
5. Use **transitions, duration, and `on('start')` / `on('end')`** highlighting.
6. A **sorting dropdown** that changes order through animated updates.

Rules clarified for reproducible tests

- Preserve `allRecords` in **source-file order**, and maintain an **active subset** separately.
- Sorting changes *visual order*, not the membership of the active subset.
- A valid province has a nonempty **unique identifier/name** and a finite, nonnegative GRDP value. Do not silently drop duplicate keys: investigate them.
- After removing and re-adding a record, it appears **once**. Do not allow fewer than 1 or more than all valid records.
- Recompute the quantitative axis from **currently displayed** data when membership changes. Explain whether its domain includes zero (normally yes for bar-length comparisons).
- Use stable keys based on a unique field (not the row's displayed position).

4. Setup — before THINK (included in instructor intro)

Create a project:

```
lab04-d3/
├── index.html
├── chart.js
├── style.css
└── NOTES.md              # THINK, VERIFY, ADAPT, AI record
```

Include **D3 v7** in your page, and use **only D3** as the charting library. Other ordinary browser APIs and CSS are allowed. Run using a local web server; inspect DevTools **Console** and **Network** if the CSV fails to load.

Your page must display: **Lab 4** (browser title), your **names/student IDs**, **Data Science and Data Visualization**, **Lab 4 — D3.js: Transition, Update, Enter & Exit**, and the original handout's academic-integrity statement: "This is all my own work. I did not copy the code from any other source." For this AI-permitted lab, interpret the statement as an **individual responsibility**

for submitted work: identify any AI-generated assistance in `NOTES.md`; do not claim to have personally typed or invented every line. Ask the instructor if institutional wording must remain unchanged.

Starter snippets (not a finished chart)

Data loading + exploration:

```
const url = 'https://tungth.github.io/data/vn-provinces-data.csv';
d3.csv(url)
  .then(rows => {
    console.log('CSV headers:', rows.columns);
    console.table(rows.slice(0, 3));
    // TODO: decide which real columns mean province ID/name and GRDP.
    // TODO: convert numbers carefully and validate the values.
  })
  .catch(err => console.error('CSV failed:', err));
```

Update pattern with stable identity:

```
const bars = chart.selectAll('rect.bar')
  .data(visibleRecords, d => d.id);  // id must uniquely identify a province

// TODO: design the enter, update and exit behaviors.
// TODO: animate axes and labels, not just rectangles.
```

Event handler (modern D3):

```
d3.select('#sort-select').on('change', function (event) {
  const criterion = event.target.value;
  // TODO: sort a COPY for display; keep original data intact.
});
```

Do not copy field names from online answers without examining this CSV. D3 v7 uses a Promise for `d3.csv()` and `event` as an explicit listener parameter; old examples using `d3.event` or a `(error, data)` CSV callback may be incompatible.

5. THINK — AI OFF — 20 minutes

Write short answers in `NOTES.md` **before** opening an AI assistant. Discuss each answer together.

T1 — Predict the join. Starting with active keys `[A, B, C]`, update to `[C, A, D]`. Which item should **enter**, which should **exit**, and which should **update**? Does order change object identity? Why does `.data(records, (d,i) => i)` risk the wrong result?

T2 — Decide on state. Draw a small diagram distinguishing: (a) raw CSV rows, (b) validated `allRecords`, (c) included/active records, (d) sorted records used for drawing. Explain why sorting should not mutate `allRecords`.

T3 — Define control semantics. After the user adds 3 provinces, sorts descending by GRDP, then clicks **Remove**, which province disappears? Explain how your design implements this rule.

T4 — Predict failures. Identify at least 3 risks: for example wrong key, changing chart height, stale axis, missing labels, parsing decimal commas, rapid repeated clicks, duplicate records, zero GRDP or unavailable data.

T5 — Choose the visual encoding. Identify the categorical axis, quantitative axis and units, the rule for label placement, and how the chart height changes with the number of bars. Why would beginning the bar axis far above zero mislead a reader?

THINK checkpoint: Be ready to explain your data-state sketch, chosen key and one predicted bug to the instructor.

6. BUILD — AI ON — 55 minutes

Build ONE interactive page. Work in small increments and test after every change.

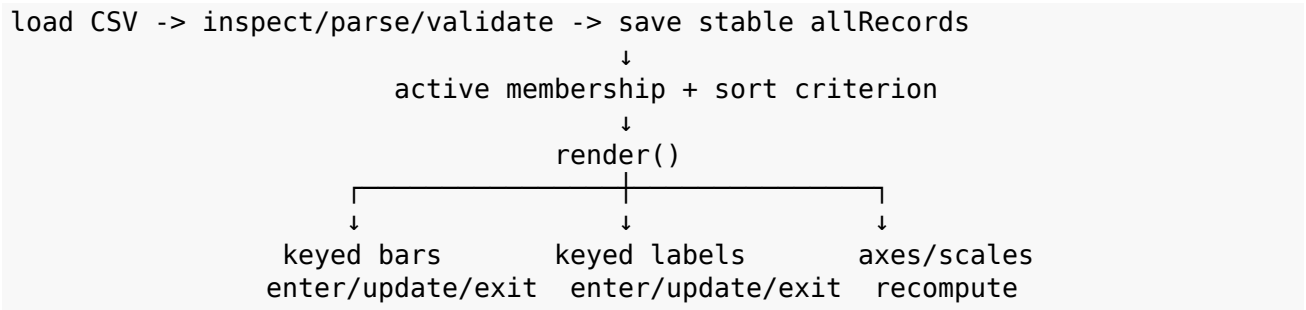
B1 — Load, inspect and clean (10 min). Display the real CSV headers and sample rows in DevTools. Choose the correct source fields for province identity/name and GRDP. Convert the GRDP field to a JavaScript number. Check `Number.isFinite` and explain any excluded records. Check for duplicate identities. Record the selected field names and units in `NOTES.md`. If the source fails, report the error; do **not** silently substitute fabricated provincial values.

B2 — Draw the initial chart (15 min). Show **20** valid provinces. Use margins, a linear x-scale, a band y-scale, an x-axis with ticks and units, province labels (axis or adjacent labels), and value labels on the right. Keep chart width fixed and the chart height appropriate for the number of displayed bars. Use a reusable `render()` or equivalent function.

B3 — Add and Remove (15 min). Add buttons. Add inserts the next valid source record; Remove removes the last included record. Use the D3 keyed join to handle enter/update/exit. Update **bars, province labels, value labels, chart height, scales and axes**. Disable or explain controls at minimum/maximum limits. Smoothly animate **enter** and **exit**, and remove exiting DOM nodes when transitions finish.

B4 — Sort and animate (15 min). Implement a dropdown with at least: (a) original source order; (b) province name A–Z; (c) GRDP high–low. Sorting should reorder the same visual items, not create duplicates. Set a meaningful `.duration(...)`, animate bars and text positions, and use `.on('start', ...)` and `.on('end', ...)` to highlight a change. Explain how rapid clicks during a transition are handled.

The following structure is suggested, not prescribed:



7. VERIFY — AI ON — 20 minutes

Record **at least five** evidence-based tests in `NOTES.md`:

Test	Expected result	Actual observation / evidence	Pass?
V1. Initial state	Exactly 20 unique bars and matching labels		
V2. Add then remove	Counts 20->21->20; original set restored		
V3. Sort	Same identities, different order, no duplicates		
V4. Boundary	Cannot add past last record or remove past minimum		
V5. Axis and values	Ticks, bar widths, labels agree after membership change		

Debug one intentionally introduced defect. Choose one: (D1) key by array index, (D2) omit exit removal, (D3) leave the x-axis domain fixed when maximum changes, or (D4) update bars but not value labels. Before fixing, document: *expected -> observed -> likely cause -> fix -> retest*. The test should be observable, not merely "AI says it looks good."

AI review prompt (recommended): "Review my D3 keyed update logic. Identify bugs when sorting, adding/removing records, or clicking rapidly. Do not rewrite the entire chart. Propose concrete tests." Verify at least one suggestion; note what you accepted or rejected.

8. ADAPT — AI ON — 15 minutes

New request: The user accidentally clicks **Remove** and wants an **Undo Remove** button.

- Undo restores **only the most recently removed record** in its prior membership position; it must not duplicate an already active province.
- A second Undo without another removal must do nothing (disable the button or show a clear message).
- Sorting still works before and after Undo.
- The chart, axis and labels update through the existing render/update mechanism, without rebuilding the page from scratch.

Test these sequences: **Remove->Undo**; **Remove->Undo->Undo**; **Sort->Remove->Undo**. Explain in 2–3 sentences what changed in your state model and why your initial design was or was not easy to extend.

9. GROUP CHECK — 15 minutes (course-wide)

At the instructor's request, both partners should be able to:

1. Explain where the **unique key** is defined and how one object remains the same when sorting.
2. Demonstrate an **enter, update, exit** sequence and identify the corresponding DOM change.
3. Explain one **verification test**, including what would fail if an axis or label was stale.
4. Make a small change on the existing code (e.g., duration 700->300 ms, an alternate sorting direction, or revised add/remove limit) without blindly replacing the full program.

The instructor may check selected pairs during earlier phases as well; 15 minutes does **not** mean 15 minutes separately for every group.

10. Submission

Submit ONE ZIP file per pair through the LMS:

```
Lab04_Group_XX.zip
├── index.html
├── chart.js
├── style.css
└── NOTES.md
```

Include: THINK answers and state sketch; your source-field/unit notes; **five verification tests** with outcomes; the documented defect/fix; your Undo explanation/tests; and a short AI collaboration record (2–3 entries with prompt, suggestion, decision, test).

Important: Do not submit merely screenshots or AI explanations instead of working HTML/JS/CSS. Provide enough instructions to run the solution on a local web server. The original data URL must remain visible in the source and source note. If an offline copy is used for contingency, it must be the authentic source file with attribution.

Assessment rubric

Evidence	Points
THINK: update/state model, key and expected failures	20
BUILD: correct chart, controls, transitions and labels	20
VERIFY: tests, induced defect, root cause and retest	20
ADAPT: Undo and three documented sequences	15
GROUP CHECK: explanation, short change and both members' ownership	25
Total	**100**

11. Optional homework — one week, maximum 3 hours per pair

Add a toggle that temporarily displays **Top 10 by GRDP** without corrupting the underlying active-membership state. Restore the previous view and demonstrate that Add/Remove/Undo still behave predictably. Submit one before/after screenshot and a brief design rationale. This is optional unless assigned by the instructor; it is not part of the 100-point in-class rubric.

References

- Original DSDV Lab 04: *Transition, Update, Enter & Exit* (local course handout)
- Scott Murray, *Interactive Data Visualization for the Web*, Chapter 9 (*Updates, Transitions, and Motion*). Note: historical chapter examples may use older D3 syntax.
- https://d3js.org/ • https://d3js.org/d3-selection/joining • https://d3js.org/d3-transition
- Course dataset: https://tungth.github.io/data/vn-provinces-data.csv